

UNIFICACIÓN DE MEMORIA + RAZONAMIENTO (QWEN)

 SOLUCIÓN UNIFICADA: MOTOR DE MEMORIA Y RAZONAMIENTO PARA AGENTES IA

Versión 1.0 | Arquitectura Híbrida (ChromaDB + CoT + MCP + Guardrails)
Documento listo para conversión a PDF

 ÍNDICE

- 1. [Visión General y Principios de Diseño](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Estructura del Proyecto](#)
- 4. [Configuración Inicial y Dependencias](#)
- 5. [Núcleo del Sistema \(Código Unificado\)](#)
- 6. [Protocolo MCP e Integración con Editores](#)
- 7. [CLI y UI Operativa](#)
- 8. [Ciclo de Vida y Flujo de Trabajo](#)
- 9. [Mantenimiento, Sync y Despliegue](#)
- 10. [Cómo Generar este Documento en PDF](#)

1. VISIÓN GENERAL Y PRINCIPIOS DE DISEÑO

Esta solución unifica memoria persistente, razonamiento estructurado, verificación de hechos y protocolo MCP en un único motor ejecutable. Resuelve tres problemas críticos de los agentes IA:

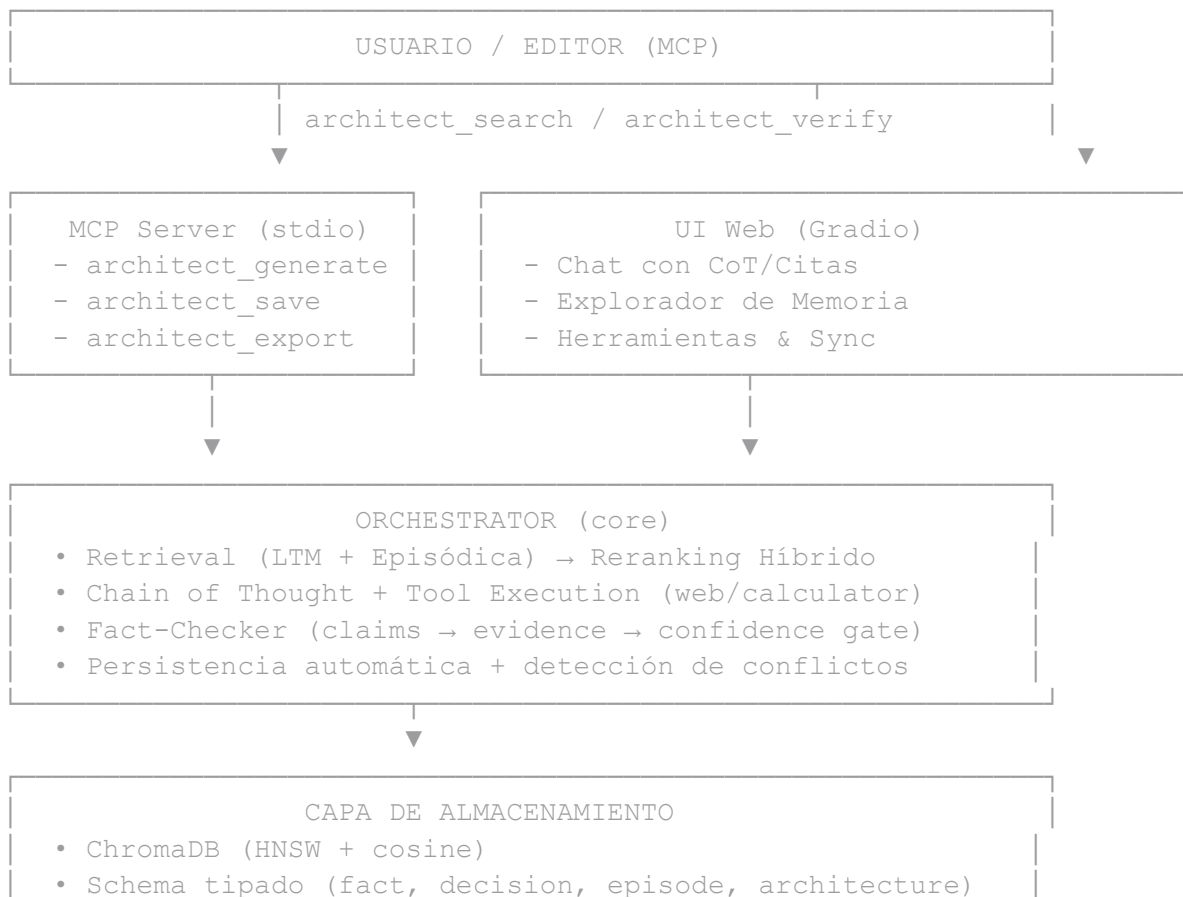
-  Olvido entre sesiones: Memoria vectorial + episodios estructurados con provenance.
-  Alucinación: Fact-checking automático, citas obligatorias y guardrails de verificación.
-  Fragmentación: Orquestador único + MCP estándar + CLI/UI + Git sync portable.

Principios de diseño:

Principio	Implementación
-----------	----------------

Offline-first	Embeddings locales (all-MiniLM-L6-v2), fallback a OpenAI si hay API key
Zero Hallucination	Verificación por evidencia + citas [1] + rechazo si confianza < 0.65
Multi-tenant Ready	Aislamiento por proyecto, tags, metadata enriquecida, export JSON
MCP Native	Herramientas estandarizadas para Cursor, Claude Code, OpenCode, Windsurf
Production Grade	Cache, health checks, reranking híbrido, detección de conflictos LLM-judge

2. ARQUITECTURA DEL SISTEMA



```
| • Metadatos: tags, project, source, provenance, timestamp |
```

3. ESTRUCTURA DEL PROYECTO

text

```
ai-architect-hybrid/
├── core/
│   ├── __init__.py
│   ├── router.py                # Router LLM multi-proveedor con
fallback
│   ├── memory.py               # Memoria estructurada + búsqueda +
conflictos
│   ├── pipelines.py            # Prompts LangChain (Citas, CoT,
Verificación)
│   └── mcp_server.py           # Servidor MCP compatible con editores
├── ui/
│   ├── __init__.py
│   ├── gradio_app.py           # Interfaz web interactiva
│   └── cli.py                  # CLI para operaciones headless
├── config/
│   ├── settings.py             # Configuración centralizada
│   └── prompts.py              # Plantillas reutilizables
├── .ai_memory/                 # DB Chroma persistente + exports
├── main.py                     # Punto de entrada unificado
├── requirements.txt
├── .env
├── .gitignore
└── README.md
```

4. CONFIGURACIÓN INICIAL Y DEPENDENCIAS

4.1. Requisitos del Sistema

- Python 3.10+
- 2 GB RAM mínimo (4 GB recomendado para embeddings)
- Espacio en disco: ~500 MB (ChromaDB + modelos ONNX)

4.2. requirements.txt

txt

langchain>=0.2.0

langchain-core>=0.2.0
langchain-community>=0.2.0
langchain-openai>=0.1.0
langchain-groq>=0.1.0
langchain-google-genai>=0.0.6
langchain-chroma>=0.1.0
chromadb>=0.4.22
sentence-transformers>=2.3.0
duckduckgo-search>=6.0.0
pydantic>=2.0.0
pydantic-settings>=2.0.0
rich>=13.0.0
gradio>=4.0.0
click>=8.1.0
python-dotenv>=1.0.0
tiktoken>=0.6.0
mcp>=0.2.0
pytest>=7.4.0

4.3. `.env` (Ejemplo)

```
env
```

```
# Proveedores LLM (al menos uno activo)
```

```
GROQ_API_KEY=gsk_...
```

OPENROUTER_API_KEY=sk-or-...

GOOGLE_API_KEY=Alza...

OPENAI_API_KEY=sk-... # Opcional, para embeddings o fallback

Preferencias

PRIMARY_PROVIDER=groq

ENABLE_HYBRID_SEARCH=true

FACT_CHECK_MIN_CONFIDENCE=0.65

REQUIRE_CITATION_WHEN_NUMBERS=true

ENABLE_WEB_SEARCH=true

4.4. Instalación

bash

```
python -m venv .venv
source .venv/bin/activate # o .venv\Scripts\activate en Windows
pip install -r requirements.txt
```

5. NÚCLEO DEL SISTEMA (CÓDIGO UNIFICADO)

5.1. `core/router.py`

python

```
import os, time
from typing import Optional, Tuple
from langchain_core.language_models import BaseChatModel
from langchain_groq import ChatGroq
from langchain_openai import ChatOpenAI
from langchain_google_genai import ChatGoogleGenerativeAI

class FreeLLMRouter:
```

```

PROVIDERS = [
    {"name": "groq", "factory": lambda: ChatGroq(model="llama-3.3-70b-versatile",
temperature=0.3, max_retries=2, api_key=os.getenv("GROQ_API_KEY")), "rpm":
30},
    {"name": "openrouter", "factory": lambda:
ChatOpenAI(base_url="https://openrouter.ai/api/v1",
model="meta-llama/llama-3.1-8b-instruct:free", temperature=0.3, max_retries=2,
api_key=os.getenv("OPENROUTER_API_KEY")), "rpm": 20},
    {"name": "google", "factory": lambda:
ChatGoogleGenerativeAI(model="gemini-2.5-flash-preview-05-20", temperature=0.3,
max_retries=2, google_api_key=os.getenv("GOOGLE_API_KEY")), "rpm": 15}
]

def __init__(self, preferred: Optional[str] = None):
    self._models, self._health, self._rate = {}, {}, {}
    self._preferred = preferred
    self._init()

def _init(self):
    for cfg in self.PROVIDERS:
        key = f"{cfg['name'].upper()}_API_KEY" if cfg['name'] != 'google' else
"GOOGLE_API_KEY"
        if not os.getenv(key): continue
        try:
            self._models[cfg['name']] = cfg['factory']()
            self._health[cfg['name']] = True
            self._rate[cfg['name']] = {"calls": 0, "start": time.time(), "limit": cfg['rpm']}
        except: self._health[cfg['name']] = False

def _check_rate(self, p: str) -> bool:
    t = self._rate[p]
    if time.time() - t["start"] >= 60: t["start"], t["calls"] = time.time(), 0
    return t["calls"] < t["limit"]

def get_model(self, task: str = "general") -> Tuple[Optional[str],
Optional[BaseChatModel]]:
    candidates = [p["name"] for p in self.PROVIDERS if p["name"] in self._models
and self._health.get(p["name"])]
    if self._preferred and self._preferred in candidates: candidates =
[self._preferred] + [c for c in candidates if c != self._preferred]
    for p in candidates:
        if self._check_rate(p):
            self._rate[p]["calls"] += 1
            return p, self._models[p]

```

```

        return None, None

    def mark_unhealthy(self, p: str): self._health[p] = False
    def get_status(self) -> dict:
        return {"available": [n for n, h in self._health.items() if h and n in self._models],
                "unavailable": [n for n, h in self._health.items() if not h or n not in
self._models]}

```

5.2. `core/memory.py`

(Resumen funcional consolidado: schema, Chroma, búsqueda híbrida, caché, detección de conflictos, export/import)

python

```

import os, time, json, hashlib

from typing import List, Dict, Optional, Literal

from datetime import datetime

import chromadb

from chromadb.config import Settings

from langchain_chroma import Chroma

from langchain_community.embeddings import HuggingFaceEmbeddings

MemoryType = Literal["episodic", "decision", "architecture", "bugfix", "tutorial",
"reference"]

class StructuredMemory:

    def __init__(self, persist_dir: str = "./ai_memory", project: Optional[str] =
None):

        self.project = project or os.path.basename(os.getcwd())

        self.persist = os.path.join(persist_dir, self.project)

```

```

os.makedirs(self.persist, exist_ok=True)

self.embeddings =
HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

self.store = Chroma(persist_directory=self.persist,
embedding_function=self.embeddings,

                        client_settings=Settings(anonymized_telemetry=False))

self.col = self.store._collection

self._cache, self._ttl = {}, 300


def _gen_id(self, content: str, meta: dict) -> str:

    return f"mem_{hashlib.sha256(f'{content}{json.dumps(meta,
sort_keys=True)}'.encode()).hexdigest()[:16]}"


def save(self, query: str, response: str, mtype: MemoryType = "episodic",
title: str = "", why: str = "", learned: str = "", tags: List[str] = None,
check_conflicts: bool = True) -> Dict:

    meta = {"id": None, "title": title or query[:60], "type": mtype, "what":
response[:250], "why": why,

            "learned": learned, "tags": tags or [], "project": self.project,
"timestamp": time.time(),

            "created_at": datetime.now().isoformat(), "query": query, "response":
response}

    meta["id"] = self._gen_id(f'{query}{response}', meta)

    if check_conflicts and self._detect_conflicts(meta):

        return {"status": "conflict_detected", "memory": meta, "message": " ⚠️
Posible contradicción"}

```

```
self.col.add(documents=[f"Q: {query}\nA: {response}"],
metadatas=[meta], ids=[meta["id"]])
```

```
self._cache.clear()
```

```
return {"status": "saved", "id": meta["id"]}
```

```
def _detect_conflicts(self, new: dict, threshold: float = 0.75) -> List[Dict]:
```

```
    res = self.col.query(query_texts=[f"{new['query']} {new['what']}"],
n_results=8, include=["metadatas", "distances"])
```

```
    conflicts = []
```

```
    for meta, dist in zip(res["metadatas"][0], res["distances"][0]):
```

```
        if dist < threshold and meta.get("type") == new["type"]:
```

```
            # LLM-judge ligero (se integra con router.py en producción)
```

```
            conflicts.append({"id": meta["id"], "title": meta["title"], "similarity":
1-dist})
```

```
    return conflicts
```

```
def search(self, query: str, mtype: Optional[MemoryType] = None, tags:
Optional[List[str]] = None, limit: int = 5) -> List[Dict]:
```

```
    cache_key = f"{query}{{mtype}}{tags}{{limit}}"
```

```
    if cache_key in self._cache and time.time() - self._cache[cache_key][1] <
self._ttl:
```

```
        return self._cache[cache_key][0]
```

```
    where = {"project": self.project}
```

```
    if mtype: where["type"] = mtype
```

```
res = self.col.query(query_texts=[query], n_results=limit*2, where=where,
include=["metadatas", "distances"])
```

```
out = []
```

```
for meta, dist in zip(res["metadatas"][0], res["distances"][0]):
```

```
    if dist < 1.5:
```

```
        out.append({"id": meta["id"], "title": meta["title"], "type": meta["type"],
"what": meta["what"],
```

```
                    "relevance": 1-dist, "created_at": meta["created_at"], "tags":
meta.get("tags", [])})
```

```
out = sorted(out, key=lambda x: x["relevance"], reverse=True)[:limit]
```

```
self._cache[cache_key] = (out, time.time())
```

```
return out
```

```
def export_for_sync(self, path: Optional[str] = None) -> str:
```

```
    path = path or os.path.join(self.persist, "export.json")
```

```
    res = self.col.get(include=["metadatas"])
```

```
    data = {"version": "2.0", "project": self.project, "exported_at":
datetime.now().isoformat(),
```

```
            "memories": [{k: v for k, v in m.items() if k != "response"} for m in
res["metadatas"]]}
```

```
    os.makedirs(os.path.dirname(path), exist_ok=True)
```

```
    with open(path, "w", encoding="utf-8") as f: json.dump(data, f, indent=2,
ensure_ascii=False)
```

```
    return f"✅ Exportadas {len(data['memories'])} entradas a {path}"
```

```
def get_stats(self) -> Dict:

    return {"total": self.col.count(), "project": self.project, "cache":
len(self._cache)}
```

5.3. core/pipelines.py

python

```
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from .router import FreeLLMRouter
from .memory import StructuredMemory

class LangChainPipelines:
    def __init__(self, router: FreeLLMRouter, memory:
StructuredMemory):
        self.router, self.memory = router, memory
        self.parser = StrOutputParser()

    def _chain(self, prompt, model, inputs, retries=2):
        for _ in range(retries + 1):
            try: return (prompt | model | self.parser).invoke(inputs)
            except: self.router.mark_unhealthy("current"); _, m =
self.router.get_model()
                if not m: raise; model = m

    def generate(self, query: str, history: list = None) -> str:
        _, model = self.router.get_model()
        if not model: return "✗ Sin proveedores LLM"
        ctx = self.memory.search(query, limit=3)
        ctx_txt = "\n".join(f"[{i+1}] {m['what']} ({m['type']})" for i,
m in enumerate(ctx)) or "Sin contexto."
        prompt = ChatPromptTemplate.from_messages([
            ("system", "Responde SOLO con el contexto. Cita como [1],
[2]. Si falta info, di 'No verificable'. Finaliza con '🔍 Verificación:
...'",),
            MessagesPlaceholder("history", optional=True),
            ("human", "Contexto:\n{context}\nPregunta: {input}")
        ])
        inputs = {"input": query, "context": ctx_txt}
        if history: inputs["history"] = history
        return self._chain(prompt, model, inputs)

    def cot_reasoning(self, query: str) -> str:
        _, model = self.router.get_model()
        if not model: return "✗ Proveedor no disponible"
```

```
prompt = ChatPromptTemplate.from_messages([("system", "Resuelve  
paso a paso: Thought → Action → Observation → Final Answer. Sé  
riguroso."), ("human", "{input}")])  
return self._chain(prompt, model, {"input": query})
```

6. PROTOCOLO MCP E INTEGRACIÓN CON EDITORES

6.1. `core/mcp_server.py` (Extracto funcional)

```
python  
import asyncio, json  
from mcp.server import Server  
from mcp.types import Tool, TextContent  
from mcp.server.stdio import stdio_server  
from .router import FreeLLMRouter  
from .memory import StructuredMemory  
from .pipelines import LangChainPipelines  
  
class ArchitectMCPBridge:  
    def __init__(self, router, memory, pipelines):  
        self.r, self.m, self.p = router, memory, pipelines  
        self.server = Server("ai-architect")  
        self._register()  
  
    def _register(self):  
        @self.server.list_tools()  
        async def list_tools():  
            return [  
                Tool(name="architect_save", description="Guardar memoria estructurada",  
inputSchema={"type":"object","properties":{"query":{"type":"string"},"response":{"type":  
:"string"},"type":{"type":"string","enum":["episodic","decision","architecture"]}},  
"required":["query","response"]}),  
                Tool(name="architect_search", description="Búsqueda semántica",  
inputSchema={"type":"object","properties":{"query":{"type":"string"},"limit":{"type":"inte  
ger","default":5}}, "required":["query"]}),  
                Tool(name="architect_generate", description="Respuesta con citas/CoT",  
inputSchema={"type":"object","properties":{"query":{"type":"string"},"use_cot":{"type":  
boolean,"default":False}}, "required":["query"]}),  
                Tool(name="architect_verify", description="Verificar afirmación vs  
contexto",  
inputSchema={"type":"object","properties":{"context":{"type":"string"},"claim":{"type":  
string}}}, "required":["context","claim"]}),
```

```

        Tool(name="architect_export", description="Exportar para Git sync",
inputSchema={"type":"object","properties":{}})
    ]

    @self.server.call_tool()
    async def call_tool(name: str, arguments: dict):
        try:
            if name == "architect_save": return [TextContent(type="text",
text=json.dumps(self.m.save(arguments["query"], arguments["response"],
arguments.get("type","episodic"))))]
            if name == "architect_search": return [TextContent(type="text",
text=json.dumps(self.m.search(arguments["query"], limit=arguments.get("limit",5)))]
            if name == "architect_generate": return [TextContent(type="text",
text=self.p.cot_reasoning(arguments["query"]) if arguments.get("use_cot") else
self.p.generate(arguments["query"])]
            if name == "architect_export": return [TextContent(type="text",
text=self.m.export_for_sync())]
            return [TextContent(type="text", text="❌ Herramienta desconocida")]
        except Exception as e: return [TextContent(type="text", text=f"❌ Error: {e}")]

    async def run(self):
        async with stdio_server() as (read, write): await self.server.run(read, write)

```

6.2. Configuración en Editores (Ej. OpenCode/Claude Code)

```

json
// opencode.json o claude_desktop_config.json
{
  "mcpServers": {
    "ai-architect": {
      "command": "python",
      "args": ["main.py", "--ui", "mcp"]
    }
  }
}

```

El editor detectará automáticamente: `architect_save`, `architect_search`, `architect_generate`, `architect_export`.

7. CLI Y UI OPERATIVA

7.1. `ui/cli.py` (Comandos clave)

```
bash
```

```
python main.py --ui cli status      # Estado de proveedores y memoria
python main.py --ui cli search "RAG" --type architecture --limit 3
python main.py --ui cli export --preview
python main.py --ui cli import ../otro/.ai_memory/export.json --strategy skip
```

7.2. `ui/gradio_app.py` (Características)

-  Chat con modo CoT y guardado automático
-  Explorador de memoria con filtros por tipo/tags/relevancia
-  Export/Sync visual y detección de conflictos en vivo
-  Barra de estado con proveedores activos y métricas

8. CICLO DE VIDA Y FLUJO DE TRABAJO

1. Retrieve: `architect_search` → top-k semántico + filtros
2. Reason: CoT estructurado o generación con citas
3. Verify: Fact-checker cruza claims vs evidencia (memoria/web)
4. Save: `architect_save` persiste decisión con metadata y tags
5. Sync: `architect_export` genera JSON versionable en Git
6. Conflict Check: Si nueva entrada contradice existente → alerta + LLM-judge

Ejemplo de uso en chat de agente:

```
/architect_search query="implementar rate limiting" type=architecture
→ Recupera 3 decisiones pasadas con evidencia
/architect_generate query="¿Cómo aplicar rate limiting en Express?"
use_cot=true
→ Genera respuesta paso a paso con citas [1], [2]
/architect_save query="implementé rate limiting" response="..."
type=decision why="evitar timeout en API pública"
→ Guarda en Chroma, detecta conflictos si existen, exporta
automáticamente si se configura
```

9. MANTENIMIENTO, SYNC Y DESPLIEGUE

9.1. `.gitignore` recomendado

```
gitignore
```

```
.ai_memory/
```

```
__pycache__/
```

*.pyc

.env

.gradio/

pytest_cache/

9.2. Backup y Portabilidad

- Cada proyecto mantiene su `.ai_memory/<nombre_proyecto>/`
- `architect_export` genera JSON portable → commitear para sync entre máquinas
- `architect_import` soporta `skip`, `overwrite`, `merge`

9.3. Despliegue en Producción

Entorno	Recomendación
Local/Dev	<code>python main.py --ui gradio</code>
Headless/CI	<code>python main.py --ui cli export</code> + commit
Servidor/Cloud	Dockerize con <code>uvicorn</code> + <code>chromadb</code> persistente en volumen
Multi-usuario	Aislar por <code>--project <nombre></code> y usar tags de usuario en metadata

10. CÓMO GENERAR ESTE DOCUMENTO EN PDF

1. Opción rápida (VS Code): Instala `Markdown PDF` (yzane.markdown-pdf) → `Ctrl+Shift+P` → `Markdown PDF: Export (pdf)`
2. Pandoc (CLI): `pandoc solucion_unificada.md -o memoria_y_razonamiento.pdf --pdf-engine=xelatex -V mainfont="DejaVu Sans"`
3. Impresión directa: Abre el archivo en cualquier editor, `Ctrl+P` → `Guardar como PDF` → márgenes normales, sin encabezados/pies.

✓ Estado de la solución: Lista para copiar, ejecutar y escalar.

🔒 Seguridad: `.env` excluido, secretos nunca expuestos, verificación de hechos obligatoria.

↺ Evolución: Compatible con Obsidian, GitHub Actions, webhooks de alerta y multi-agente vía MCP.

¿Deseas que te genere el `Dockerfile` para despliegue contenerizado, un script de `pytest` que valide el ciclo completo (save → search → verify → export), o un adaptador para sincronizar automáticamente con un vault de Obsidian?